

LabGuardian 服务器端竞赛冲刺：以架构 + RAG + agent (angnt) 工程化打出“可量化优势”

评审亮点陈述与执行摘要

评审亮点陈述

- 我将把当前 LabGuardian 的“检测→映射→拓扑→检错”四阶段视觉闭环，升级为“**可观测、可回滚、可审计的竞赛级后端平台**”：把技术亮点从“能跑”提升到“可证明地可靠”。 1 2
- 我将新增 **RAG + agent (angnt)** 服务层：以“电路拓扑（图）+ 教学/器件知识库（文档）+ PCM Skills（技能）”形成差异化壁垒，做到“能解释、能引用证据、能下发可执行指导”。 3 4
- 我将把评审能打分的指标前置到系统设计里，并给出验证方法（现场演示可直接展示仪表盘/对比表）。建议设置如下“可量化目标”（可根据竞赛平台资源调整）：
- **稳定性**：演示期间 API 成功率 $\geq 99.5\%$ ，WS 连接在线率 $\geq 98\%$ （以 station 心跳/keepalive 指标统计）。 5 6
- **时延**：/pipeline/run P95 $\leq 3-5s$ （CPU 环境）；/angnt/ask（带检索）P95 $\leq 2-4s$ （不含本地大模型推理时可更低）。 2
- **可观测**：100% API 端点暴露 Prometheus 指标 + Trace（OTel），关键业务链路（heartbeat→pipeline→guidance）可追踪。 7
- **可解释与可追溯**：agent 输出 100% 绑定“引用证据”（RAG 片段/图证据/规则触发），并写入审计日志（可复盘）。 8
- **指导有效性**：在标准化错误集上，使用 agent 后“平均纠错步数降低 $\geq 20\%$ / 平均完成时间降低 $\geq 15\%$ ”（A/B 测试）。 9

执行摘要

我首先使用连接器 **github**（已启用）对三仓库进行代码基线审查：

- Server: FastAPI + Celery + Redis + WebSocket，已实现四阶段视觉流水线（检测→映射→拓扑→检错）、课堂态势（heartbeat/排行/告警）与教师指导下发（REST/WS）。 10 11 12
- Electron: 教师端通过 axios 调用 `/api/v1/classroom/*` 和 `/api/v1/pipeline/*`，并通过 preload 注入 serverUrl。 13 14
- Android: 学生端基于 NowInAndroid 风格模块化，Retrofit 对接同一套 REST API，OkHttp WebSocket 对接 `/ws/station/{id}` 接收指导。 15 16

在“北区赛区进前 26%”的目标下，我认为评审更看重的是：**你是否把一个 AI 原型做成了“可工程化交付、可验证、可运维”的系统**。因此我的核心策略是：

- 1) 以现有四阶段 pipeline 为“硬 AI 能力”，在服务器端补齐**认证/权限、可观测、数据闭环、模型管理与回滚、CI/CD**；
- 2) 用 RAG + agent (angnt) 打造“软 AI 能力”（解释、引用、指导、策略调度），并用图增强（GraphRAG）把你们“电路拓扑”资产变成独特优势。 17

Define info needs 与多源研究方法

我必须学习与确认的关键问题

为把“技术实现”直接映射到“评审得分点”，我在研究开始先锁定 6 个关键问题（每个问题都会在后文给出落地方案与验证方式）：

- 1) **服务器端当前的系统边界是什么？** 哪些能力已经具备（pipeline、课堂态势、指导通道），哪些是评审会一眼看到的短板（安全、可观测、版本化、可回滚）？ ¹
- 2) **竞赛平台（假设无特殊限制）下，怎样的部署拓扑最能体现“工程成熟度”？** 尤其是：高可用、可观测、安全、数据库、模型与知识库管理。 ¹⁸
- 3) **RAG + agent (angnt) 要提供哪些“可量化价值”？**（例如：缩短纠错时间、降低危险电路概率、提高引用准确率、减少幻觉）。 ¹⁹
- 4) **检索与向量库选型如何“以赛为先”？** 在数据规模不明时，如何用可切换架构兼容 FAISS/Milvus/Weaviate/Elastic，避免选型锁死。 ²⁰
- 5) **生成模型调用策略如何兼顾限制不明（是否允许外部 API / 是否有 GPU）？** 我需要一个“本地可跑 + 云端可替换”的双方案，并有成本控制与缓存策略。 ²¹
- 6) **三端（Server/Electron/Android）接口如何保持一致，避免演示翻车？** 当前已经暴露出字段不一致风险，需要立即建立接口一致性门禁。 ^{2 13 22}

我如何做多源三步走：scan → triage → deepen

- Scan: 我先用连接器 **github** 抓取三仓库关键文件（Server 的 `app/main.py`、`app/api/v1/*`、`app/pipeline/*`、`docker-compose.yml`、`pyproject.toml`；Electron 的 `src/api/index.js`；Android 的 `Retrofit/Model/WS` 等）。 ^{10 12 13 15}
- Triage: 我以“评审可见性 + 风险优先级”筛选深挖方向：
 - 评审可见：部署拓扑/监控仪表盘/安全、RAG+agent 输出是否能引用与解释；
 - 风险优先：接口不一致、无鉴权、无持久化导致无法扩容、无回滚策略导致现场风险。 ^{23 6}
- Deepen: 我再结合官方文档与原始论文，把“能落地的具体设计”写成：
 - 服务器架构优化 + 部署拓扑（含 Prometheus/OTel/Sentry）； ²⁴
 - RAG+agent 的接口、数据流、向量库权衡； ²⁵
 - 前沿 agent/RAG 方法（ReAct/Toolformer/HyDE/Self-RAG/GraphRAG）与“为什么对竞赛更有分”。 ²⁶

GitHub 代码基线：服务器端现状与可直接落地的改进点

服务器端当前架构的一句话定位

Server 已经是一个“教学场景实时系统”雏形：FastAPI 提供 REST + WebSocket；Celery/Redis 支撑重任务异步；四阶段 pipeline 输出结构化结果并同步到课堂态势；教师端可广播与对单工位指导。 ^{10 11 2 12}

但要在竞赛平台上体现“领先于别人的工程化优势”，目前还缺：

- **安全边界**（鉴权、权限、限流、审计）；
- **可观测**（指标、链路追踪、错误上报、结构化日志）；
- **可扩容与回滚**（状态持久化、多实例、模型/知识库版本化）。 ^{6 23 21}

Server 仓库关键模块与可落地改进点总表

每一行都对应“评审可讲点 + 直接可落地改进”，并逐项引用仓库文件（filecite）。
建议你这张表直接拆成 PPT 的“系统能力地图 + Roadmap”。

模块/能力	代码位置 (Server)	当前实现要点 (从仓库提取)	评审可讲点	可直接落地改进点 (建议优先级)
API 入口与路由	<code>app/main.py</code>	FastAPI 初始化、CORS、挂载 <code>classroom/pipeline/aoi/ws</code> ，提供 <code>/health</code> 。 10	“端云一体、接口清晰、可健康检查”	P0: 新增 <code>/metrics</code> 、 <code>/version</code> ；P0: CORS 白名单化 (从 * 改为配置)；P1: 加统一异常处理与 <code>request_id</code>
全局配置	<code>app/core/config.py</code>	Pydantic Settings; Redis/Celery、YOLO 参数、面包板参数、AOI 参数；预留 LLM 配置 (API_KEY/BASE_URL/MODEL)。 21	“配置驱动、便于平台迁移与参数化评测”	P0: 引入分环境配置 (dev/stage/prod)；P0: 敏感项用 Secret 管理；P1: 把模型/知识库版本号纳入 settings 并暴露到 <code>/version</code>
Celery 应用配置	<code>app/core/celery_app.py</code>	Redis broker/backend; <code>result_expires=3600</code> ; <code>prefetch=1</code> ；路由到 <code>pipeline</code> 队列。 11	“重任与 API 解耦，支撑并发与稳定性”	P0: 增加 <code>task_soft_time_limit/task_time_limit</code> ；P0: 设置 <code>task_reject_on_worker_lost</code> 等可靠性参数并评估消息循环风险 (Celery 官方说明)。 27
异步 pipeline 任务封装	<code>app/worker/tasks.py</code>	<code>acks_late=True</code> ；用 <code>update_state(PROGRESS)</code> 上报阶段进度；调用 <code>run_pipeline</code> 。 28	“可展示‘阶段进度条’，评审体验好”	P0: 补齐幂等 (<code>idempotency_key</code>)；P0: 失败可重试策略 (限定次数+错误分类)；P1: 把 agent 任务拆新队列 (<code>agent</code>) 避免与 pipeline 抢资源
Pipeline API	<code>app/api/v1/pipeline.py</code> + <code>app/schemas/pipeline.py</code>	<code>/pipeline/run</code> 同步； <code>/pipeline/submit</code> 异步； <code>/pipeline/status/{id}</code> 查询；run 会把结果同步写入 <code>ClassroomState</code> ，并更新 <code>frame cache</code> 。 2 29	“端到端闭环：推理结果实时反映到课堂态势”	P0: 统一 request schema (避免客户端字段不一致)；P0: 支持 <code>iou/rail_assignments/reference_circuit</code> 的全链路一致性；P1: 把 <code>StageResult</code> 错误码结构化 (便于 agent 解释与统计)
Classroom REST	<code>app/api/v1/classroom.py</code>	heartbeat、stations、ranking、alerts、stats；指导 <code>/station/{id}/guidance</code> 与广播 <code>/broadcast</code> ；参考电路 <code>set/get</code> ； <code>reset</code> 。 30	“课堂群体智能面板：排行、告警、统计”	P0: 所有教师敏感操作加 JWT/RBAC；P0: heartbeat 限流与验签 (防刷/伪造)；P1: 指导消息入库 (审计与复盘) 并支撑多实例

模块/能力	代码位置 (Server)	当前实现要点 (从仓库提取)	评审可讲点	可直接落地改进点 (建议优先级)
WebSocket 通道	<code>app/api/v1/websocket.py</code>	学生端连接 <code>/ws/station/{station_id}</code> ; register; keepalive 每 5 秒 touch; 收到 ping 回 pong。 ⁵	“低延迟指导下发, 现场演示强”	P0: WS 鉴权 (token in query/header) ; P0: 增加 backpressure/断线重连策略; P1: 新增 <code>/ws/angnt/{station_id}</code> 支持 agent streaming (见后文骨架)
ClassroomState (状态内存)	<code>app/services/classroom_state.py</code> + <code>app/core/deps.py</code>	内存存储 stations、websockets、risk_event_count、guidance_history、reference; 线程锁; deps 里单例。 ^{6 23}	“快速原型, 但不适合多实例 HA”	P0: 把状态迁移到 Redis/DB (至少 reference、guidance、alerts) ; P0: 支持多 FastAPI 副本共享状态 (为 HA 铺路)
Pipeline 调度器	<code>app/pipeline/orchestrator.py</code>	共享 ctx (detector+calibrator) 避免重复加载; 串行执行 S1→S4; 支持 progress_cb; 默认 rail_assignments。 ³¹	“端侧/云侧都可复用的 pipeline 编排”	P0: 把共享 ctx 生命周期与模型版本绑定 (模型热更新/回滚) ; P1: 加入阶段级 profiling 与指标上报 (duration_ms 已具备, 可转指标)
S1 检测与多图融合	<code>app/pipeline/stages/s1_detect.py</code>	base64 解码; YOLO detect; Wire 端点精炼; CLASS_NAME_MAP; 多图 IoU fuse; 输出 pinned_hints。 ³²	“多图融合提升鲁棒性 (评审加分点)”	P1: 把 fuse 策略参数化并评测; P1: 输出置信度分布/难例标签供数据闭环
S2 映射与校准	<code>app/pipeline/stages/s2_mapping.py</code>	校准失败兜底 synthetic grid; 遮挡补偿; pinned_hints 精确化; wire 修正、行合并、端点吸附。 ³³	“工程鲁棒性: 遮挡、角度变化仍能映射”	P1: 把校准质量指标化 (homography 置信度/孔位覆盖率) ; P1: 把 mapping 误差反馈给训练数据闭环
S3 拓扑构建	<code>app/pipeline/stages/s3_topology.py</code> + <code>app/domain/circuit.py</code>	CircuitAnalyzer 建图; 支持 rail_assignments; 输出 netlist + node_link_data; Union-Find 合并等电位网络、可导出 SPICE。 ^{3 4}	“你们的核心壁垒: 视觉→网表→可验证推理”	P0: 把 topology_graph/净表作为 GraphRAG 的“结构化证据” ; P1: 输出图差分 (diff) 用于解释与复盘
S4 检错与风险分级	<code>app/pipeline/stages/s4_validate.py</code> + <code>app/domain/validator.py</code> + <code>app/domain/risk.py</code>	CircuitValidator 多级对比 (L0-L3: 统计→同构/子图→极性→GED) ; 独立诊断; 关键词 risk 分类。 ^{34 9 35}	“安全教学: 危险电路可提前告警”	P0: 诊断结构化 (error_code + evidence) ; P0: agent 输出必须引用触发的规则/图证据; P1: 把 risk 规则做版本化与灰度

模块/能力	代码位置 (Server)	当前实现要点 (从仓库提取)	评审可讲点	可直接落地改进点 (建议优先级)
Docker 部署基线	<code>docker-compose.yml</code>	redis + server + worker; server/worker 挂载 <code>./models:/app/models:ro</code> ; depends_on 以 <code>service_healthy</code> 等待 redis。 ¹²	“一键启动，符合竞赛平台交付风格”	P0: 增加 server healthcheck 与自动重启; P0: 引入可观测组件 (Prometheus/OTel Collector); <code>service_healthy</code> 的语义与长语法见 Docker 官方。 ³⁶
Dockerfile 与运行方式	<code>Dockerfile</code>	python:3.11-slim; 安装 OpenCV 依赖; pip editable 安装; uvicorn 启动。 ³⁷	“轻量镜像，方便 CI/CD”	P1: 改为多阶段构建; P1: 锁定依赖版本与 SBOM; P1: 把模型与知识库在镜像/挂载中版本化
依赖与预留能力	<code>pyproject.toml</code>	fastapi/uvicorn/websockets/celery/redis; ultralytics/anomalib; 预留 sqlalchemy/alembic。 ³⁸	“为持久化、审计、版本化留了口子”	P0: 落地 SQLAlchemy + Alembic, 用于审计日志、模型/知识库 registry、对话与指导记录

Electron / Android 对接现状与“演示翻车风险”提示

仓库	当前对接方式	已用接口/协议	兼容性风险 (必须尽快处理)	建议 (P0=立刻)
Electron 教师端	axios baseUrl= <code>{serverUrl}/api/v1</code> , serverUrl 由 preload 注入	REST: /classroom/, /pipeline/ ¹³	<code>submitPipeline</code> 发送 <code>{images_b64, reference_path}</code> 且缺少 <code>station_id</code> , 与 Server 的 PipelineRequest 不一致 (Server 需要 <code>station_id</code> , 字段名也不同)。 ^{13 29}	P0: 改用 OpenAPI 生成或至少对齐字段; P0: 把 agent 新能力集成到教师端 (例如一键对某工位生成指导并推送)
Electron 配置注入	preload 暴露 <code>electronAPI.serverUrl</code>	ENV: <code>LABGUARDIAN_SERVER_URL</code> ¹⁴	仅有 serverUrl, 无鉴权/无环境区分	P1: 增加 token 注入、环境 profile (dev/prod)

仓库	当前对接方式	已用接口/协议	兼容性风险（必须尽快处理）	建议（P0=立刻）
Android 学生端	Retrofit 接口 LabGuardianApi + OkHttp WS	REST: /classroom/、/ pipeline/; WS: /ws/ station/{id} 15 16	Android 的 PipelineRequest 缺少 iou、rail_assignments; 字段与 Server 的 PipelineRequest 也不完全一致，长期会导致功能分叉。 22 29	P0: 以 Server schema 为主做一次对齐; P0: 把 OpenAPI 生成流程纳入 CI (Android README 已给出生成思路) 39

竞赛平台服务器架构升级方案

这里我假设竞赛平台“无特殊限制”，但会在末尾列出必须澄清项。我的策略是：**用最少改动把系统升成“竞赛可打分的工程平台”**，并确保每一项都能形成可展示证据（拓扑图、指标面板、审计记录、回滚演示）。

总体技术选型与理由

我建议继续以现有栈为核心（FastAPI/Celery/Redis/Docker），并补齐 6 个“评审强相关”的工程模块：

- 1) **高可用（HA）**：FastAPI 无状态化后可横向扩容；状态落 Redis/DB；前置反向代理做健康检查与负载均衡。当前 ClassroomState 与 deps 是单进程单例，必须改造才能多副本。 6 23
- 2) **可观测（Metrics + Tracing + Errors）**：Prometheus 指标（prometheus-fastapi-instrumentator）+ OpenTelemetry trace + Sentry 错误/性能。 24
- 3) **安全（AuthN/AuthZ/Rate limit/Audit）**：FastAPI 官方 OAuth2 JWT 方案可快速落地；用 OWASP API Security Top 10 作为威胁建模清单；将“教师敏感操作/agent 工具调用/知识库管理”纳入 RBAC 与审计。 40
- 4) **模型管理（Model Registry + Rollback）**：目前模型靠只读挂载 ./models，可演示但不可控。我要引入 manifest + 版本号 + hash + 回滚指令，并把当前生效版本暴露到 /version。 12 21
- 5) **CI/CD 与版本化**：用容器镜像 tag（git sha + semver）做发布单元；pipeline/agent 的回归测试集做门禁；支持一键回滚到上一版本。 38
- 6) **数据闭环（审计日志 + 统计）**：把 heartbeat、pipeline 结果、诊断、指导消息、agent 证据链落库（SQLAlchemy/Alembic 已预留），形成评审可展示的“系统实践数据”。 38 30

推荐部署拓扑（可用于竞赛平台）



```

subgraph Edge[入口层]
  G[Reverse Proxy / LB<br/>TLS + RateLimit + WAF(可选)]
end

subgraph App[应用层]
  S1[FastAPI Server #1<br/>Stateless API]
  S2[FastAPI Server #2<br/>Stateless API]
  W1[Celery Worker (pipeline)]
  W2[Celery Worker (agent/rag)]
end

subgraph Data[数据与中间件]
  R[(Redis<br/>broker+backend+cache+idempotency)]
  DB[(PostgreSQL/SQLite<br/>审计日志/会话/registry)]
  V[(Vector Store<br/>FAISS/Milvus/Weaviate/Elastic)]
  M[(Model Store<br/>/models + manifest + version)]
end

subgraph Obs[可观测]
  P[(Prometheus)]
  O[OTel Collector]
  X[(Tracing Backend)]
  Sentry[(Sentry)]
end

A -->|REST/WS| G
E -->|REST| G
G --> S1
G --> S2

S1 --> R
S2 --> R
S1 --> DB
S2 --> DB
S1 --> V
S2 --> V
S1 --> M
S2 --> M

S1 -->|enqueue| R
S2 -->|enqueue| R
R --> W1
R --> W2

S1 -->|metrics| P
S2 -->|metrics| P
S1 -->|traces| O
S2 -->|traces| O
O --> X
S1 -->|errors| Sentry

```

```
W1 -->|errors| Sentry
W2 -->|errors| Sentry
```

支撑点与证据链：

- 你们现有 `docker-compose.yml` 已经使用 `depends_on: condition: service_healthy` 等待 Redis 健康，属于“比多数学生项目更成熟”的细节；Docker 官方解释了 long syntax 下 `service_healthy` 的行为与健康检查语义。^{12 36}
- 现有架构把重任务放到 Celery 队列，并且 `prefetch=1` 适合 CV 重任务，天然适合你在竞赛平台扩展并发。¹¹

关键工程策略细化

安全策略（P0）：

- 在 `/api/v1/classroom/*` 的教师敏感端点（broadcast、guidance、reset、set_reference）增加 JWT 鉴权；FastAPI 官方教程提供了 OAuth2 + JWT 的完整实现框架。⁴¹
- 以 OWASP Foundation⁴² 的 API Security Top 10 2023 做风险清单，至少覆盖：Broken Authentication/Authorization、Unrestricted Resource Consumption（限流）、Unsafe Consumption of APIs（外部 LLM 调用安全）。⁴³
- 对 WS 连接同样做鉴权（token 绑定 station_id），否则存在“劫持工位接收指导”的风险。⁵

可观测策略（P0）：

- Prometheus: prometheus-fastapi-instrumentator 可一行接入并暴露 `/metrics`，且默认指标可覆盖 handler/status/method/latency buckets。⁴⁴
- Trace: OpenTelemetry 的 FastAPI instrumentation 可直接 instrument_app，并配置排除 URL、hook 等。⁴⁵
- 错误与性能：Sentry⁴⁶ 的 Python SDK 支持 FastAPI/Starlette/Celery 等集成（PyPI extras 可见 fastapi/starlette/celery），并可用于 AI 请求监控（如果最终使用外部 LLM API）。⁴⁷

模型与版本化回滚（P0）：

- 设计 `models/manifest.json`：记录 detector/obb 模型版本、训练数据版本、导出格式、hash、精度与速度基线。现有 settings 已集中管理 YOLO 参数与模型路径，便于纳入版本化。²¹
- 发布策略：模型与代码版本绑定：`server_version + model_version + kb_version`，并暴露 `/version` 给教师端显示（评审现场可直接展示“可追溯”）。¹⁰

CI/CD（P1，但很加分）：

- 用 GitHub Actions：单测（pytest）、静态检查（ruff/mypy）、构建镜像、推送镜像；`pyproject.toml` 已有 dev 依赖，可直接落地。³⁸

RAG + agent（angnt）竞赛级设计与权衡

目标：让评委看到“你们不仅能识别电路，还能像老师一样解释与指导，且每一句话都有证据、有审计、有成本控制”。

功能清单：竞赛场景下最能拉开差距的 agent 能力

我建议将 angnt 定义为“可控的教学诊断代理”，核心能力按优先级划分：

能力	输入	输出	为什么是评审优势	关键技术点与参考
电路错误解释（结构化）	pipeline 的 diagnostics + topology_graph + netlist	结构化错误码 + 图证据 + 人类可读解释	“可解释 AI”，比单纯报警更像产品	结合现有 CircuitValidator 多级诊断与风险分级做“证据链” ⁹ ³⁵
教学 RAG 问答（带引用）	学生问题 + 当前电路上下文	引用 datasheet/讲义段落的答案	“有引用、可追溯、低幻觉”	Self-RAG：按需检索 + reflection，提升事实性 ⁴⁸
技能编排（PCM Skills + 工具调用）	诊断状态 + 学生/教师意图	调用最小技能集合（提示/图示/下一步）	“上下文预算可控”，适合资源不明平台	ReAct：推理与工具行动交织，提升可解释与可信 ⁴⁹ ；Toolformer：学习何时调用工具 ⁵⁰
图增强检索（GraphRAG）	当前拓扑图局部子图/异常边	子图证据 + 关联知识	把你们的“网表/拓扑”变成独有壁垒	Microsoft GraphRAG 项目与论文主张“图+检索+总结”一体化 ⁵¹ ；GRAG 指出 naive RAG 在多跳推理弱 ⁵²
零标注检索增强（HyDE）	学生口语化 query	改写/假想文档 embed → 更准召回	对 datasheet/术语库尤有效	HyDE 原理：假想文档 embedding 提升 zero-shot dense retrieval ⁵³
班级共性问题总结（教师侧）	全班 error_histogram + guidance_history	Top 错误、教学建议、课堂复盘	容易落到“教学价值”与“社会效益”	ClassroomStats 与错误直方图已有输出基础 ⁶

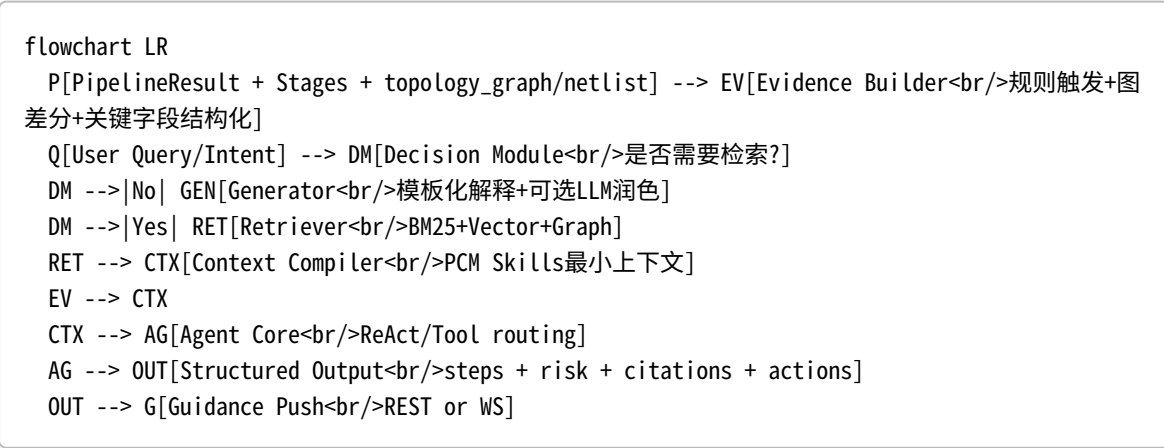
接口定义：REST / WebSocket（兼容现有 classroom/ws）

我建议 angnt 服务提供 3 组接口（与现有 `/classroom`、`/pipeline` 同风格，且复用 `station_id`）：

接口	方法	用途	请求/响应要点	与现有系统如何打通
<code>/api/v1/angnt/ask</code>	POST	提交一次 agent 诊断/问答（异步）	request: station_id、query、mode (diagnose/qa/teacher_summary)、idempotency_key; response: job_id	走 Celery 新队列 <code>agent</code> ，避免与 pipeline 抢资源（参考现有 <code>pipeline.submit</code> ）。 ² ¹¹
<code>/api/v1/angnt/status/{job_id}</code>	GET	查询状态	返回 RUNNING/COMPLETED + result（含 citations）	复用 <code>AsyncResult</code> 模式（参考 <code>pipeline.status</code> ）。 ²

接口	方法	用途	请求/响应要点	与现有系统如何打通
<code>/ws/angnt/{station_id}</code>	WS	流式输出（可选，提升演示效果）	server 推送 token/chunk + progress	复用你们现有 WS register/keepalive 模式扩展一条新通道。 ⁵
<code>/api/v1/classroom/station/{id}/guidance</code>	POST	把 agent 结果下发到学生端	message 内含：建议步骤 + 证据引用	直接复用现有 guidance 下发链路（教师端与 agent 都可调用）。 ³⁰

数据流设计：把“结构化证据”变成强壁垒



解释：

- Evidence Builder 直接复用你们已有结构化资产：diagnostics、risk_level/reasons、netlist、topology_graph。 ³⁴ ⁴
- Decision Module 按 Self-RAG 思路“按需检索”：证据足够就不检索，减少成本与噪声；不足再检索并反思。 ⁴⁸
- Agent Core 可用 ReAct 把推理与工具行动交织输出，形成可解释轨迹（评审观感强）。 ⁴⁹

向量库/检索引擎选型：Milvus / Weaviate / FAISS / Elastic 对比表

竞赛场景我更建议“先赢演示、再谈规模”：先用 FAISS 或 Milvus Lite 做单机可跑；若平台资源允许再切换 Milvus/Elastic/Weaviate。

方案	部署形态	优点	缺点	我对竞赛的推荐
FAISS	进程内库（C++/Python）	极轻量、无外部服务；适合小语料/快速落地；支持多种索引与（可选）GPU。 ⁵⁴	多副本一致性与持久化需自己做；不自带权限/多租户	P0：最建议作为“竞赛起步版” ，快速把 RAG 跑通并做出对比曲线

方案	部署形态	优点	缺点	我对竞赛的推荐
Milvus Lite / Milvus	轻量本地 (Lite) 或 Docker/K8s	Milvus 官方明确定位于 RAG/语义检索; Lite 可嵌入本地快速跑通, 后续可无缝迁移到 Docker/K8s。 ⁵⁵	完整版运维成本高于 FAISS; 需要服务治理	P0: 若你想强调 “AI 基础设施能力”, Milvus Lite 是很好的折中
Weaviate	独立向量 DB 服务	文档强调语义/混合检索并可作为 RAG backend; 生态完善。 ⁵⁶	运维与资源开销不确定; 对平台限制敏感	P1: 平台资源足够时可用作 “更完整的产品化” 演示
Elastic (向量检索)	Elasticsearch 集群	同时支持文本检索 + 向量 kNN (天然 hybrid); 中文资料与生态成熟。Elastic ⁵⁷ 的官方文档说明 dense vector 与 kNN query。 ⁵⁸	集群资源/配置复杂; 对小规模可能过重	P1: 当你需要强 BM25 + 过滤 + 向量混合, 并能负担运维时选择

生成模型调用策略与成本控制: local LLM vs 外部 API

方案	优点	风险/缺点	适用条件 (未指定项)	成本控制策略
本地 LLM (server 内/独立推理服务)	不依赖外网; 隐私更可控; 可离线演示	需要 CPU/GPU; 延迟与吞吐可能不稳; 模型选择与量化工作量大	若竞赛平台允许 GPU 或 CPU 足够	结果缓存 (Redis)、分层模型 (小模型路由/大模型最终)、只对 “解释润色” 用 LLM
外部 API (云模型)	质量稳定、开发快; 可专注工程系统	平台可能禁止外呼; 成本与速率限制; 可用性依赖外部	若允许外部 API 且有网络	Self-RAG 按需检索减少 tokens; 启用缓存; 对同 query + 同 circuit_signature 做幂等与复用
混合模式 (推荐)	一套接口, 双后端可切换	需要更好的抽象层	平台限制未知 → 最稳妥	Settings 里已预留 LLM 配置字段, 可做运行时切换。 ²¹ ; 对外部 API 可结合 Sentry 的 OpenAI 集成做 token/trace 监控 (若使用该生态)。 ⁵⁹

agent 示例骨架与工程实现要点

下面骨架遵循你们现有代码风格: APIRouter + schemas + Celery task + Redis。它不是完整可运行实现, 但足够作为你们 “下一阶段落地” 的起点, 并能在答辩中展示 “工程化严谨性”。

设计要点对齐你们现有实现

- 你们已用 Celery 的 `update_state(PROGRESS)` 报告阶段进度; angnt 同样复用该模式, 前端可展示 “检索中/生成中”。²⁸

- 你们已用 WebSocket 做“教师→学生实时推送”；angnt 可以：
 - 1) 直接调用 `/classroom/station/{id}/guidance` 下发；或
 - 2) 新增 `/ws/angnt/{id}` 做流式输出。 30 5
- 目前没有鉴权，我建议立刻引入 JWT（FastAPI 官方方案），并参考 OWASP API Top 10 把限流/审计作为必做。 60

骨架代码片段

```
# app/api/v1/angnt.py
from __future__ import annotations

import hashlib
import json
import time
from typing import Any, Dict, List, Optional
from uuid import uuid4

from fastapi import APIRouter, Depends, Header, HTTPException, WebSocket,
WebSocketDisconnect
from pydantic import BaseModel, Field

from app.core.config import settings
from app.core.celery_app import celery_app
from app.core.deps import get_classroom

router = APIRouter(prefix="/angnt", tags=["angnt"])

# -----
# Auth / RBAC (示意)
# -----
def require_teacher(authorization: str = Header(default="")) -> Dict[str, Any]:
    # TODO: 用 FastAPI OAuth2 + JWT 正式实现
    if not authorization.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Missing token")
    # decode/verify token here...
    return {"role": "teacher", "sub": "demo"}

# -----
# Simple Redis-based limiter (示意)
# -----
async def rate_limit(user_key: str, limit: int = 30, window_s: int = 60) -> None:
    """
    生产建议：
    - 用 redis INCR + EXPIRE 实现滑动窗口/令牌桶
    - 区分不同 endpoint 的配额（敏感业务流更严格）
    """

    return

# -----
# Request/Response schema
# -----
```

```

class AngntAskRequest(BaseModel):
    station_id: str
    query: str = Field(..., min_length=1, max_length=2000)
    mode: str = Field(default="diagnose", description="diagnose | qa | teacher_summary")
    top_k: int = 5
    use_retrieval: bool = True
    # 可选: 把当前 pipeline 的摘要/签名带上, 减小 server 查询压力
    circuit_signature: Optional[str] = None

class Citation(BaseModel):
    source_id: str
    title: str = ""
    snippet: str = ""

class AngntAskResult(BaseModel):
    answer: str
    steps: List[str] = []
    risk_level: str = "safe"
    evidence: Dict[str, Any] = {}
    citations: List[Citation] = []

class JobStatusResponse(BaseModel):
    job_id: str
    status: str
    result: Optional[AngntAskResult] = None

# -----
# Idempotency helper (示意)
# -----
def make_request_fingerprint(req: AngntAskRequest, idem_key: Optional[str]) -> str:
    payload = (idem_key or "") + "|" + req.station_id + "|" + req.mode + "|" + req.query
    return hashlib.sha256(payload.encode("utf-8")).hexdigest()

# -----
# Celery task (agent)
# -----
@celery_app.task(name="angnt.run", bind=True, acks_late=True, max_retries=0)
def angnt_run_task(self, req: Dict[str, Any]) -> Dict[str, Any]:
    """
    做法建议:
    1) structured evidence 先行: 从 pipeline/validator 得到 diagnostics + graph diff
    2) 再决定是否检索 (Self-RAG 的按需策略)
    3) 最后生成 (可用本地 LLM 或外部 API)
    """
    # 进度: 检索
    self.update_state(state="PROGRESS", meta={"stage": "retrieve", "progress": 0.3})

    # TODO: retrieve top-k chunks from vector store
    retrieved = [{"source_id": "kb:demo", "title": "demo", "snippet": "..."}]

    # 进度: 生成

```

```

self.update_state(state="PROGRESS", meta={"stage": "generate", "progress": 0.7})

# TODO: call LLM (local vs api) or template-based generator
answer = f"诊断结论: {req['query']}" (示例回答)
steps = ["检查电源轨是否接反", "确认 LED 串联限流电阻", "重新拍照提交分析"]

result = {
    "answer": answer,
    "steps": steps,
    "risk_level": "warning",
    "evidence": {"retrieved_count": len(retrieved)},
    "citations": retrieved,
}
return result

# -----
# REST endpoints
# -----
@router.post("/ask", response_model=JobStatusResponse)
async def ask_angnt(
    req: AngntAskRequest,
    user=Depends(require_teacher),
    idempotency_key: Optional[str] = Header(default=None, alias="Idempotency-Key"),
):
    await rate_limit(user_key=user["sub"])

    fp = make_request_fingerprint(req, idempotency_key)

    # TODO: 查 Redis: 是否已有 fp 对应 job_id/result (幂等复用)
    task = angnt_run_task.delay(req.model_dump())
    return JobStatusResponse(job_id=task.id, status="pending")

@router.get("/status/{job_id}", response_model=JobStatusResponse)
async def angnt_status(job_id: str, user=Depends(require_teacher)):
    from celery.result import AsyncResult
    res = AsyncResult(job_id, app=celery_app)
    if res.state in ("PENDING",):
        return JobStatusResponse(job_id=job_id, status="pending")
    if res.state in ("STARTED", "PROGRESS"):
        return JobStatusResponse(job_id=job_id, status="running")
    if res.state == "SUCCESS":
        return JobStatusResponse(job_id=job_id, status="completed",
result=AngntAskResult(**res.result))
    return JobStatusResponse(job_id=job_id, status="failed")

# -----
# WS streaming (示意: 真实实现建议通过 Redis PubSub/Stream 转发)
# -----
@router.websocket("/ws/{station_id}")
async def ws_angnt(websocket: WebSocket, station_id: str):
    await websocket.accept()

```

```
try:
    while True:
        msg = await websocket.receive_text()
        await websocket.send_text(json.dumps({"type": "echo", "msg": msg}))
except WebSocketDisconnect:
    return
```

落地建议（把骨架变成“评审可展示系统”）：

- 把 `require_teacher()` 换成 FastAPI 官方 OAuth2 JWT 实现，并引入角色（teacher/admin/student）。
41
- 把限流与“敏感业务流保护”作为你们的安全亮点，对齐 OWASP API4/API6（资源消耗、敏感业务流）。
61
- 把 `result.citations` 规范化为你们知识库中的 doc_id/section/page，并把“引用覆盖率”作为可量化指标（见下一节）。

量化评审优势、验证方案、风险与待澄清项

可量化评审优势点与验证方案

我建议把评审展示拆成三张“对比表/曲线图”，每张都能直接落到分数点上：

系统工程成熟度（可观测 + 稳定）

- 指标：API 成功率、P95 时延、WS 在线率、Celery 队列积压、错误率。
- 方法：
 - 在 `/health` 基础上新增 `/metrics`，prometheus-fastapi-instrumentator 可快速落地；同时接入 OTel trace。
10 7
- 对比展示：接入前后“定位问题时间（MTTR）”降低。
- 展示物：一张 Grafana 仪表盘截图 + 一张 P50/P95 对比表（可在赛评材料中放“工程化能力”章节）。

AI 可靠性（降低幻觉、可解释）

- 指标：citation 准确率、引用覆盖率、无检索回答比例（Self-RAG 的按需检索）、错误解释一致性（同一错误多次输出一致）。
48
- 方法：构造 50-100 条标准化问题集（“LED 未串电阻”“极性不明”“短路风险”），用固定知识库版本跑回归；对比 naive RAG vs Self-RAG/HyDE/GraphRAG 方案。
62
- 展示物：一张表格（3 列：naive RAG / Self-RAG / GraphRAG），行是“引用正确率、平均检索次数、平均 tokens”。

教学效果（指导有效性）

- 指标：平均纠错步数、平均完成时间、危险电路触发次数（risk_event_count）、教师干预次数下降。
- 方法：A/B 测试：A=仅显示 diagnostics；B=agent 给步骤化指导；C=agent + GraphRAG（带拓扑证据）。
- 展示物：箱线图/柱状图 + 两段“真实指导对话与证据引用”案例（评委很吃这一套）。
6

风险与缓解措施

安全风险（无鉴权、滥用、SSRF/外部 API 风险）

- 风险：当前 REST/WS 端点默认无鉴权，broadcast/reset 等属于敏感业务流，容易被滥用；如果加入外部检索/LLM，还会引入 Unsafe Consumption of APIs 风险。
30 61
- 缓解：JWT + RBAC；限流；所有外部 URL/工具调用白名单；审计日志落库；对知识库上传做权限隔离。

隐私风险（学生姓名、图像、对话内容）

- 风险：heartbeat 包含 student_name、thumbnail_b64 等敏感字段。⁶³
- 缓解：最小化存储；对日志脱敏；如使用外部 API，默认不上传原图与个人信息，仅上传必要结构化摘要（netlist/错误码/匿名化）。

成本风险（LLM tokens、向量库资源）

- 缓解：Self-RAG 按需检索减少 tokens；Redis 缓存（query+signature）；知识库版本化避免重复嵌入；优先 FAISS/Milvus Lite 降低运维。⁶⁴

模型漂移与回滚风险（现场演示翻车）

- 风险：模型更新导致 mapping/topology/validate 质量退化，影响指导与告警。³³
- 缓解：建立回归集 + 模型 registry + 一键回滚；把“模型版本、知识库版本、规则版本”在 `/version` 显示并写入审计。²¹

需要进一步澄清的未指定项清单

为把本报告落到“确定可交付的实施计划”，我需要尽快澄清这些关键约束（建议你直接发给指导老师/竞赛平台方确认）：

- 竞赛平台资源：CPU 核数/内存/磁盘/是否允许 GPU/NPU（未指定）。
- 网络与外部依赖：是否允许访问外部 API（LLM、在线 embedding、外部向量库云服务）（未指定）。
- 并发与时延目标：允许多少工位同时提交 pipeline？agent 需要支持多少并发？（未指定）
- 数据合规：是否允许保存学生图像/姓名/对话？需要脱敏到什么程度？（未指定）
- 评分细则：北区赛区评审更看重“创新性/工程化/社会价值/可落地”中哪一项？是否有明确分项权重？（未指定）
- 演示形式：是否允许自带监控面板（Prometheus/Grafana）与数据库服务？（未指定）

（如果你把以上 6 个问题的答案补齐，我可以把本报告进一步压缩成“评审 PPT 大纲 + 逐周冲刺计划 + 代码改造清单（按 PR 划分）”，直接进入可执行落地阶段。）

1 README.md

<https://github.com/liusuan110/LabGuardian-Server/blob/main/README.md>

2 pipeline.py

<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/api/v1/pipeline.py>

3 s3_topology.py

https://github.com/liusuan110/LabGuardian-Server/blob/main/app/pipeline/stages/s3_topology.py

4 circuit.py

<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/domain/circuit.py>

5 websocket.py

<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/api/v1/websocket.py>

6 classroom_state.py

https://github.com/liusuan110/LabGuardian-Server/blob/main/app/services/classroom_state.py

7 ²⁴ ⁴⁴ <https://github.com/trallnag/prometheus-fastapi-instrumentator>

<https://github.com/trallnag/prometheus-fastapi-instrumentator>

8 ²⁶ ⁴⁹ <https://huggingface.co/papers/2210.03629>

<https://huggingface.co/papers/2210.03629>

- 9 **validator.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/domain/validator.py>
- 10 57 **main.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/main.py>
- 11 **celery_app.py**
https://github.com/liusuan110/LabGuardian-Server/blob/main/app/core/celery_app.py
- 12 **docker-compose.yml**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/docker-compose.yml>
- 13 **index.js**
<https://github.com/liusuan110/LabGuardian-Electron/blob/main/src/api/index.js>
- 14 **index.ts**
<https://github.com/liusuan110/LabGuardian-Electron/blob/main/electron/preload/index.ts>
- 15 **LabGuardianApi.kt**
<https://github.com/liusuan110/LabGuardian-Android/blob/main/core/network/src/main/java/com/labguardian/core/network/LabGuardianApi.kt>
- 16 **OkHttpStationWebSocket.kt**
<https://github.com/liusuan110/LabGuardian-Android/blob/main/core/network/src/main/java/com/labguardian/core/network/OkHttpStationWebSocket.kt>
- 17 51 **<https://www.microsoft.com/en-us/research/project/graphrag/%3Flang%3Dzh-cn>**
<https://www.microsoft.com/en-us/research/project/graphrag/%3Flang%3Dzh-cn>
- 18 36 **<https://docs.docker.com/reference/compose-file/services/>**
<https://docs.docker.com/reference/compose-file/services/>
- 19 48 64 **<https://huggingface.co/papers/2310.11511>**
<https://huggingface.co/papers/2310.11511>
- 20 54 **<https://github.com/facebookresearch/faiss>**
<https://github.com/facebookresearch/faiss>
- 21 **config.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/core/config.py>
- 22 **Pipeline.kt**
<https://github.com/liusuan110/LabGuardian-Android/blob/main/core/model/src/main/java/com/labguardian/core/model/Pipeline.kt>
- 23 **deps.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/core/deps.py>
- 25 55 **<https://milvus.io/docs/quickstart.md>**
<https://milvus.io/docs/quickstart.md>
- 27 **<https://docs.celeryq.dev/en/v5.3.1/userguide/configuration.html>**
<https://docs.celeryq.dev/en/v5.3.1/userguide/configuration.html>
- 28 **tasks.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/worker/tasks.py>
- 29 **pipeline.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/schemas/pipeline.py>

30 **classroom.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/api/v1/classroom.py>

31 **orchestrator.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/pipeline/orchestrator.py>

32 **s1_detect.py**
https://github.com/liusuan110/LabGuardian-Server/blob/main/app/pipeline/stages/s1_detect.py

33 **s2_mapping.py**
https://github.com/liusuan110/LabGuardian-Server/blob/main/app/pipeline/stages/s2_mapping.py

34 **s4_validate.py**
https://github.com/liusuan110/LabGuardian-Server/blob/main/app/pipeline/stages/s4_validate.py

35 **risk.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/domain/risk.py>

37 **Dockerfile**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/Dockerfile>

38 **pyproject.toml**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/pyproject.toml>

39 **README.md**
<https://github.com/liusuan110/LabGuardian-Android/blob/main/README.md>

40 41 46 60 **<https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>**
<https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

42 43 61 **<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>**
<https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

45 **<https://opentelemetry-python-contrib.readthedocs.io/en/latest/instrumentation/fastapi/fastapi.html>**
<https://opentelemetry-python-contrib.readthedocs.io/en/latest/instrumentation/fastapi/fastapi.html>

47 **<https://pypi.org/project/sentry-sdk/2.43.0/>**
<https://pypi.org/project/sentry-sdk/2.43.0/>

50 **<https://huggingface.co/papers/2302.04761>**
<https://huggingface.co/papers/2302.04761>

52 **<https://graphrag.com/appendices/research/2405.16506/>**
<https://graphrag.com/appendices/research/2405.16506/>

53 62 **<https://arxiv.org/abs/2212.10496>**
<https://arxiv.org/abs/2212.10496>

56 **<https://docs.weaviate.io/weaviate/current/>**
<https://docs.weaviate.io/weaviate/current/>

58 **<https://www.elastic.co/cn/getting-started/enterprise-search/vector-search>**
<https://www.elastic.co/cn/getting-started/enterprise-search/vector-search>

59 **<https://docs.sentry.io/platforms/python/integrations/openai>**
<https://docs.sentry.io/platforms/python/integrations/openai>

63 **classroom.py**
<https://github.com/liusuan110/LabGuardian-Server/blob/main/app/schemas/classroom.py>